

Processamento de Alto Desempenho (PAD)

Relatório de Trabalho 1 – Distância entre Vetores

Implementação, otimização e análise experimental de desempenho

Xavier Paulino Sebastião

Setembro de 2026

Resumo

Este trabalho avalia o cálculo da distância quadrática entre um vetor de referência e um conjunto de vetores reais e compara a implementação base com uma versão otimizada do mesmo kernel. A versão base utiliza um laço escalar. A versão otimizada emprega SIMD AVX, desenrolamento do laço, quatro acumuladores vetoriais independentes e qualificadores **restrict**, mantendo a execução em um único fluxo e as mesmas principais opções de compilação. Foram avaliados oito tamanhos de vetor, de $T = 32$ a $T = 4096$, com $N = 8192$ mantido constante na comparação. A otimização reduziu o tempo mediano em todos os tamanhos. O fator de aceleração variou aproximadamente entre 1,44 vezes e 1,80 vezes, com maior ganho em $T = 512$. Em $T = 4096$, o ganho permaneceu acima de 1, mas diminuiu para cerca de 1,59 vezes. Os resultados mostram que a reorganização do kernel aumenta a vazão e reduz o custo por elemento, embora a magnitude do ganho dependa do tamanho da entrada.

1 Introdução

O problema consiste em calcular, para cada vetor x_i de um conjunto X com N vetores, a distância quadrática em relação a um vetor de referência q . Cada vetor contém T valores reais, e a distância é definida por

$$D(q, x_i) = \sum_{j=0}^{T-1} (q_j - x_{ij})^2, \quad i = 0, \dots, N - 1. \quad (1)$$

Cada resultado é armazenado após o processamento do respectivo vetor. Mantendo N constante e variando T , a quantidade de elementos processados por execução é $N \times T$, de modo que o trabalho aritmético é esperado para crescer linearmente com T .

A primeira etapa do trabalho caracterizou o comportamento da implementação escalar para diferentes valores de T . A segunda etapa introduziu uma versão otimizada do mesmo cálculo e comparou diretamente tempo de execução, taxas de processamento, custo por elemento, fator de aceleração e variabilidade. A comparação foi realizada com múltiplas aferições, pois medições isoladas podem ser afetadas por variações transitórias da plataforma [2, 3, 8].

2 Implementações avaliadas

2.1 Versão base

A implementação de referência foi desenvolvida em C++ [7], com valores do tipo *double* e armazenamento contíguo da matriz X . Para cada vetor, o kernel percorre sequencialmente os T elementos e acumula a soma dos quadrados das diferenças. Conceitualmente, o núcleo computacional avaliado corresponde ao seguinte trecho:

```

for (size_t i = 0; i < N; ++i) {
    const double* row = X + i * T;
    double sum = 0.0;
    for (size_t j = 0; j < T; ++j) {
        const double d = q[j] - row[j];
        sum += d * d;
    }
    distances[i] = sum;
}

```

O kernel base foi mantido no mesmo programa para permitir medir as duas versões sob o mesmo procedimento experimental e evitar depender apenas de resultados obtidos em momentos diferentes.

2.2 Versão otimizada

A versão otimizada preserva o mesmo layout de dados e o mesmo cálculo matemático. A execução continua restrita a um único fluxo; portanto, o ganho não resulta do uso de vários núcleos. A otimização concentra-se no laço interno. Os ponteiros são declarados com `__restrict__`, reduzindo a possibilidade de aliasing considerada pelo compilador. Quando AVX está disponível, quatro valores `double` são processados por instrução vetorial. O laço principal é desenrolado em blocos de 16 elementos e utiliza quatro acumuladores AVX independentes, reduzindo a dependência sequencial da soma.

O trecho central pode ser representado por:

```

__m256d acc0 = _mm256_setzero_pd();
__m256d acc1 = _mm256_setzero_pd();
__m256d acc2 = _mm256_setzero_pd();
__m256d acc3 = _mm256_setzero_pd();

for (; j + 15 < T; j += 16) {
    // quatro blocos de 4 doubles
    // carga, subtracao, multiplicacao e acumulacao
}

```

Os elementos remanescentes são tratados por um bloco AVX de quatro elementos e, quando necessário, por um trecho escalar final. Assim, a implementação não exige que T seja múltiplo de 16. Como a ordem das somas difere da versão escalar, a verificação de verificação utiliza tolerância numérica, em vez de igualdade bit a bit. A compilação mantém `C++20`, `-O3` e `-march=native`, sem `-ffast-math`. O uso de SIMD e as transformações de laço estão diretamente relacionados à exploração da microarquitetura do processador [4].

2.3 Interface de execução

O tamanho do vetor T é informado como primeiro argumento posicional do executável. A saída contém uma única linha no formato

```
xavier, T, tempo_ms
```

em que o terceiro campo corresponde ao tempo de execução em milissegundos. A alocação, a inicialização dos dados e a impressão permanecem fora da região cronometrada, de modo que a medida representa exclusivamente o processamento das distâncias.

3 Metodologia experimental

3.1 Plataforma e ambiente

As aferições foram executadas em um Lenovo ThinkPad T530 equipado com processador Intel Core i5-3320M a 2,60 GHz, arquitetura x86-64, família 6, modelo 58. A CPU disponibiliza AVX, SSE4.1 e SSE4.2. A compilação foi realizada com GCC 10.5.0, usando `/usr/bin/g++`. O sistema operacional utilizado foi GNU/Linux Ubuntu 24.04.4 LTS. Os scripts de análise e geração dos gráficos foram executados no ambiente Conda `t0_vector_distance`.

3.2 Calibração da configuração experimental

Antes da comparação entre as implementações, o valor de N foi definido por meio de uma etapa de calibração realizada separadamente dos experimentos comparativos. A calibração considerou o maior valor de N , em potência de dois, cuja alocação principal para $T = 4096$ permanecesse dentro de 25% da memória `MemAvailable` observada. Com aproximadamente 1,63 GB disponíveis no momento da calibração, esse limite correspondia a aproximadamente 409 MB. Para $N = 8192$, a alocação principal estimada no maior tamanho foi de aproximadamente 269 MB, correspondente a cerca de 16,4% da memória disponível e, portanto, dentro do limite estabelecido.

A calibração também avaliou o custo da temporização com `CLOCK_MONOTONIC_RAW` [1] e verificou a menor configuração, $T = 32$. Foram realizadas 15 aferições, precedidas por três execuções de aquecimento. A mediana do tempo de execução do kernel foi de aproximadamente 231,5 μ s, enquanto a mediana do intervalo entre duas leituras consecutivas do relógio foi de 31 ns. Assim, o custo mediano associado às leituras do temporizador correspondeu a aproximadamente 0,013% do tempo mediano do kernel, permanecendo abaixo do limite de 1% definido para a calibração. Com base nesses resultados, foi adotada a configuração com $N = 8192$, *seed* 42 e CPU lógico 2 para os experimentos comparativos.

3.3 Procedimento comparativo

As versões base e otimizada foram medidas no mesmo procedimento experimental. Em cada bloco, as 16 combinações formadas por duas implementações e oito valores de T foram ordenadas de forma pseudoaleatória e determinística. Esse desenho foi adotado para reduzir a associação sistemática entre uma implementação e um momento específico da execução, uma vez que detalhes da ordem experimental podem influenciar medições de desempenho [6].

Nos resultados apresentados, $N = 8192$ permaneceu constante para as duas versões e para todos os valores

$$T \in \{32, 64, 128, 256, 512, 1024, 2048, 4096\}.$$

Cada combinação (implementação, T) foi aferida repetidamente. A mediana foi usada como medida central, enquanto a distribuição das observações foi mantida para caracterizar a variabilidade [8].

A região cronometrada foi definida para conter exclusivamente o cálculo e o armazenamento das N distâncias. Alocação, geração e inicialização dos dados, execuções de aquecimento, cálculo do checksum, escrita de arquivos e apresentação dos resultados permanecem fora do intervalo medido. A temporização utilizou `CLOCK_MONOTONIC_RAW`.

3.4 Índices de desempenho

Além do tempo de execução t , foram calculadas as taxas

$$P_v = \frac{N}{t}, \quad P_e = \frac{NT}{t}, \quad (2)$$

onde P_v representa vetores processados por segundo e P_e representa elementos processados por segundo. O custo temporal por elemento é dado por

$$C_e = \frac{t \times 10^9}{N T} \quad \text{ns/elemento.} \quad (3)$$

O fator de aceleração da versão otimizada em relação à versão base é

$$S(T) = \frac{t_{\text{base}}(T)}{t_{\text{otimizada}}(T)}. \quad (4)$$

Valores de $S(T)$ superiores a 1 indicam redução do tempo de execução. A redução percentual é expressa por

$$R(T) = 100 \left(1 - \frac{t_{\text{otimizada}}(T)}{t_{\text{base}}(T)} \right). \quad (5)$$

4 Resultados

4.1 Resumo quantitativo

A Tabela 1 reúne as principais métricas. A versão otimizada apresentou menor tempo e menor custo por elemento em todos os tamanhos avaliados. O ganho, entretanto, não foi uniforme. O menor fator de aceleração foi observado em $T = 64$, com cerca de 1,44 vezes, e o maior em $T = 512$, com aproximadamente 1,80 vezes. Em $T = 4096$, o fator de aceleração ficou em torno de 1,59 vezes.

Tabela 1: Comparação entre a versão base e a versão otimizada.

T	Tempo mediano (ms)		P_e (10^9 elem./s)		ns/elem.		Aceleração	Redução (%)
	Base	Otimizada	Base	Otimizada	Base	Otimizada		
32	0,213	0,138	1,230	1,906	0,813	0,525	1,55	35,4
64	0,468	0,324	1,120	1,618	0,893	0,618	1,44	30,8
128	1,079	0,634	0,972	1,655	1,029	0,604	1,70	41,3
256	2,235	1,357	0,938	1,546	1,066	0,647	1,65	39,3
512	4,477	2,492	0,937	1,683	1,067	0,594	1,80	44,3
1024	9,097	5,564	0,922	1,508	1,084	0,663	1,63	38,8
2048	18,364	10,463	0,914	1,604	1,095	0,624	1,76	43,0
4096	37,799	23,747	0,888	1,413	1,126	0,708	1,59	37,2

4.2 Tempo de execução

A Figura 1 mostra crescimento do tempo com T nas duas versões, mas a curva otimizada permanece abaixo da versão base em toda a faixa. Em $T = 32$, os tempos medianos são aproximadamente 0,213 ms e 0,138 ms. Em $T = 4096$, passam para cerca de 37,799 ms e 23,747 ms, respectivamente. O aumento do tamanho do vetor continua elevando o custo total, mas a otimização reduz de forma consistente a constante associada a esse crescimento.

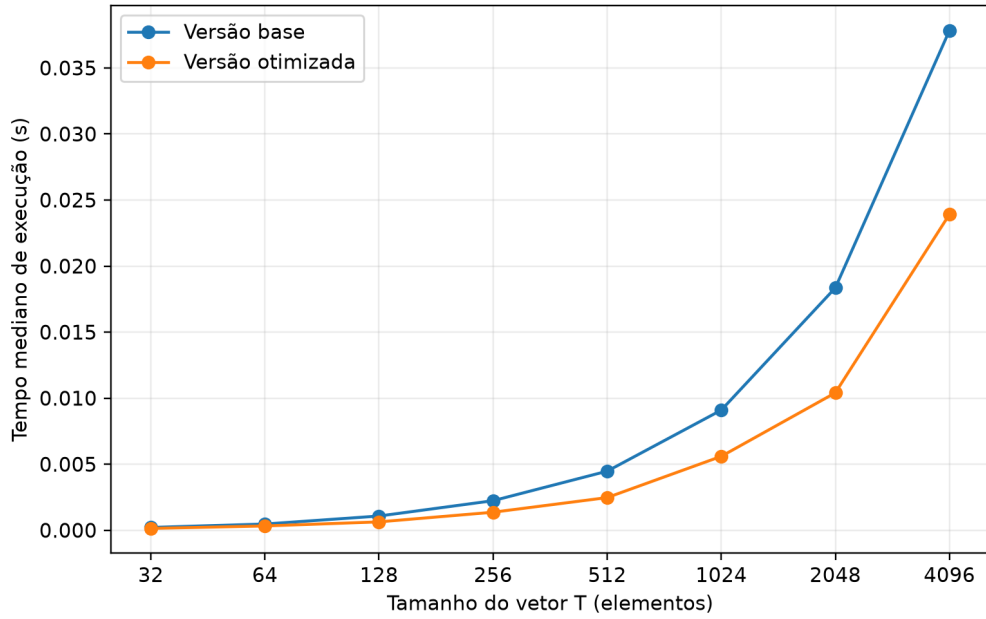


Figura 1: Tempo mediano de execução das versões base e otimizada.

4.3 Taxas de processamento

A taxa de vetores processados por segundo é maior na versão otimizada em todos os tamanhos, como mostra a Figura 2. Essa métrica diminui à medida que T cresce porque cada vetor passa a exigir mais trabalho. Por isso, sua leitura isolada não descreve a eficiência por elemento.

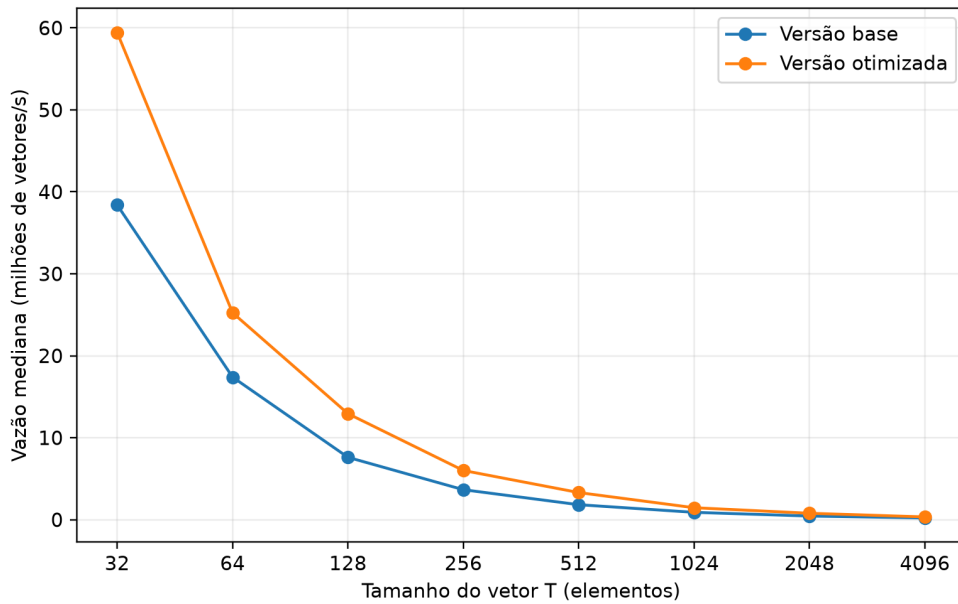


Figura 2: Vazão mediana de vetores das duas versões.

A Figura 3 apresenta a vazão em elementos por segundo. Na versão base, ela diminui de aproximadamente 1,23 bilhão de elementos/s em $T = 32$ para 0,89 bilhão em $T = 4096$. A versão otimizada alcança cerca de 1,91 bilhão de elementos/s em $T = 32$ e mantém valores entre aproximadamente 1,51 e 1,68 bilhão na maior parte da faixa intermediária, chegando a 1,41 bilhão em $T = 4096$.

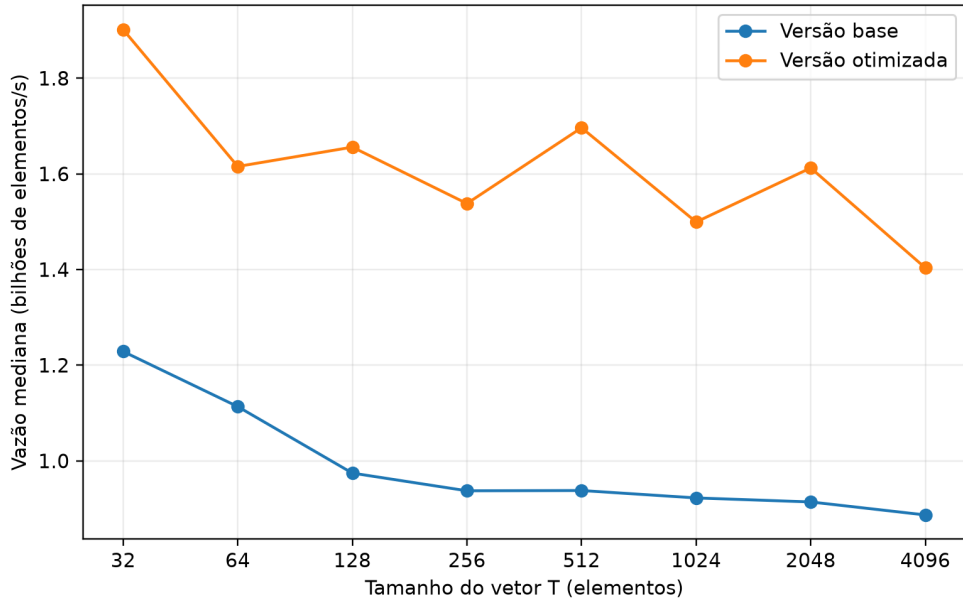


Figura 3: Vazão mediana de elementos das duas versões.

4.4 Custo por elemento

O custo por elemento fornece a mesma informação de P_e em forma inversa. Na versão base, ele aumenta de aproximadamente 0,813 ns/elemento em $T = 32$ para 1,126 ns/elemento em $T = 4096$. Na versão otimizada, varia de cerca de 0,525 ns/elemento a 0,708 ns/elemento. O menor valor da versão otimizada ocorre em $T = 32$, enquanto os tamanhos intermediários apresentam oscilações entre aproximadamente 0,59 e 0,66 ns/elemento.

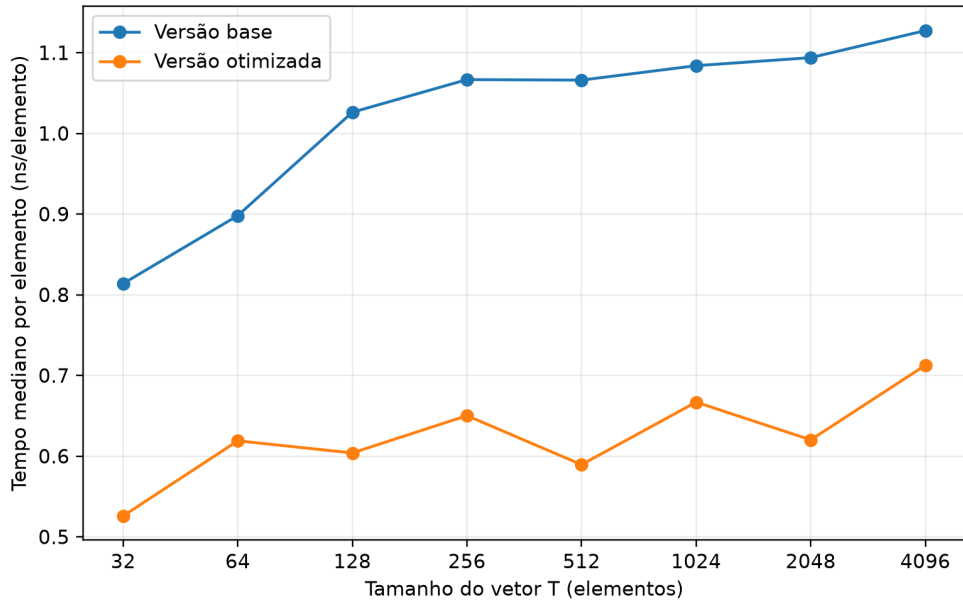


Figura 4: Tempo mediano por elemento nas duas versões.

A versão otimizada permanece mais eficiente por elemento em toda a faixa, mas a variação entre os tamanhos mostra que o benefício não pode ser descrito por uma única constante. A elevação para aproximadamente 0,708 ns/elemento em $T = 4096$ coincide com a redução do fator de aceleração nesse tamanho. Os tempos medidos, por si só, não permitem determinar se essa mudança decorre de cache, memória principal, prefetch, frequência do processador ou outro efeito

microarquitetural. Por exemplo, a infraestrutura `perf_event` do Linux pode oferecer eventos como ciclos, instruções e falhas de cache, mas esses indicadores não integram os dados avaliados [5].

4.5 Fator de aceleração

A Figura 5 resume o ganho relativo. Todos os valores permanecem acima de 1, confirmando que a versão otimizada foi mais rápida em cada tamanho avaliado. O fator varia aproximadamente de 1,44 vezes em $T = 64$ a 1,80 vezes em $T = 512$. Valores também elevados aparecem em $T = 128$ e $T = 2048$, com cerca de 1,70 e 1,76 vezes, respectivamente. Em $T = 4096$, o fator diminui para aproximadamente 1,59 vezes.

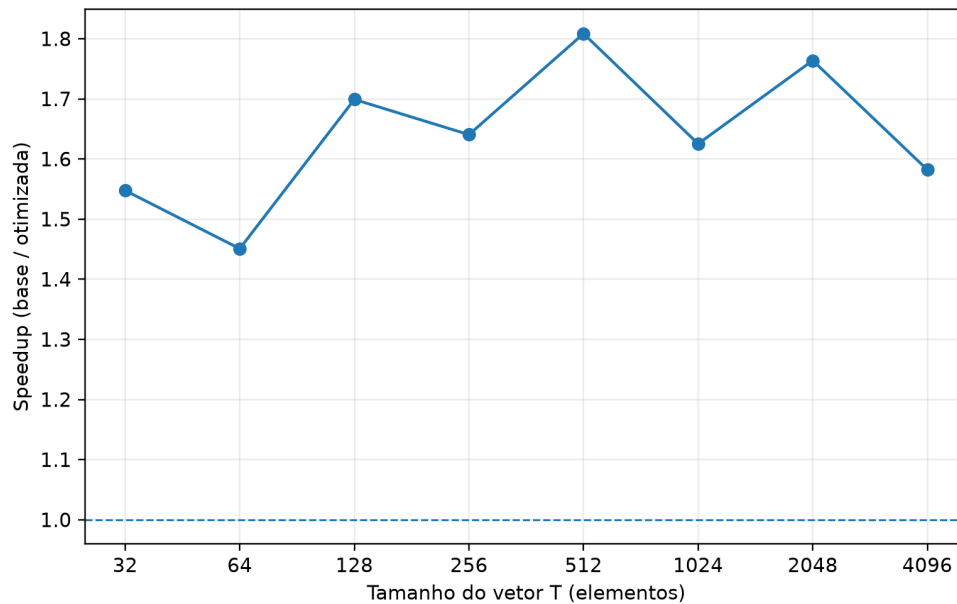


Figura 5: Fator de aceleração da versão otimizada em relação à versão base. A linha tracejada representa ausência de ganho.

4.6 Variabilidade

A Figura 6 apresenta as distribuições das aferições. A dispersão não é uniforme entre os tamanhos e cresce de forma mais visível nos maiores casos. Em $T = 2048$ aparecem observações superiores afastadas do núcleo da distribuição, e em $T = 4096$ a amplitude dos tempos é consideravelmente maior nas duas versões. As observações discrepantes foram preservadas porque não há evidência independente de erro que justifique sua remoção. Esse comportamento reforça a necessidade de considerar a distribuição das medições, e não apenas uma única execução.

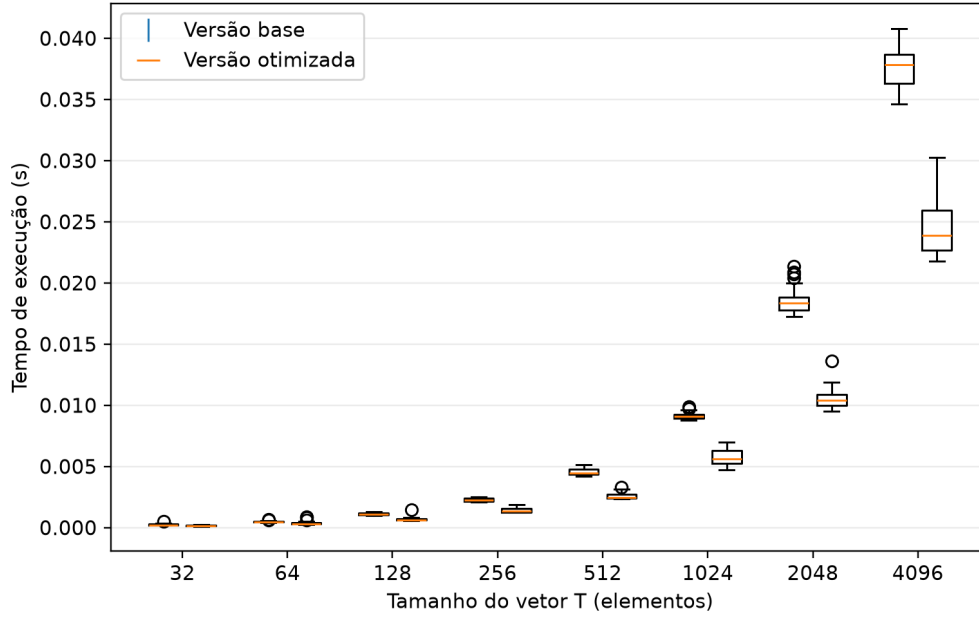


Figura 6: Distribuição dos tempos de execução das versões base e otimizada.

5 Discussão

Os resultados mostram que a otimização melhora o desempenho em toda a faixa avaliada, mas o ganho varia com T . A versão base utiliza uma única variável de acumulação, criando uma dependência entre iterações sucessivas. A versão otimizada distribui o trabalho entre quatro acumuladores AVX independentes e processa vários valores simultaneamente. Essa organização aumenta o paralelismo em nível de instrução e reduz a dependência sequencial da soma. O desenrolamento do laço também pode diminuir a frequência relativa das operações de controle.

O fator de aceleração permanece abaixo de 2 vezes em todos os tamanhos, apesar de um registrador AVX de 256 bits comportar quatro valores `double`. Esse resultado é coerente com o fato de que a capacidade de processar quatro elementos por instrução vetorial não se pode traduzir diretamente em um ganho de desempenho de 4 vezes, pois o desempenho global também é condicionado por outros fatores. O tempo total inclui carregamentos, redução dos acumuladores, controle do laço, latências das instruções e movimentação de dados. Além disso, a versão base foi compilada com `-O3 -march=native`, o que pode ter permitido ao compilador aplicar algumas otimizações e utilizar recursos de instrução disponíveis no processador.

O maior fator de aceleração ocorre em $T = 512$, com aproximadamente 1,80 vezes. Não há progressão monotônica do ganho, onde o fator diminui para cerca de 1,63 vezes em $T = 1024$, aumenta para 1,76 vezes em $T = 2048$ e alcança aproximadamente 1,59 vezes em $T = 4096$. As oscilações observadas caracterizam a dependência do ganho em relação ao tamanho da entrada, mas os dados disponíveis não permitem identificar uma causa microarquitetural específica.

A queda do ganho em $T = 4096$ coincide com menor vazão de elementos e maior custo por elemento na versão otimizada. A hierarquia de memória e o prefetch de hardware podem influenciar percursos sequenciais desse tipo [4]; entretanto, os dados disponíveis não incluem contadores de cache, tráfego de memória ou frequência instantânea. Portanto, não é possível determinar, a partir desses dados, a contribuição específica desses mecanismos para a redução observada.

A comparação foi estruturada para reduzir o efeito de diferenças temporais da máquina. As duas implementações foram medidas no mesmo procedimento e intercaladas em blocos, em vez de comparar apenas dois conjuntos independentes obtidos em momentos distintos. Esse cuidado foi considerado para cumprir as recomendações de benchmarking que enfatizam repetição, controle das condições experimentais e análise da variabilidade [2, 8, 3].

6 Conclusão

A versão otimizada apresentou menor tempo de execução em todos os tamanhos avaliados sem recorrer a paralelismo entre núcleos. O mesmo cálculo e o mesmo layout de dados foram mantidos, enquanto a otimização concentrou-se no kernel por meio de AVX, desenrolamento do laço, acumuladores independentes e `restrict`.

O fator de aceleração variou aproximadamente entre 1,44 vezes e 1,80 vezes. O maior ganho ocorreu em $T = 512$, correspondendo a redução de aproximadamente 44,3% no tempo mediano. Em $T = 4096$, o fator ficou em torno de 1,59 vezes, com redução de aproximadamente 37,2%. A versão otimizada também apresentou menor custo por elemento em todos os tamanhos, variando de aproximadamente 0,525 a 0,708 ns/elemento.

Os resultados demonstram a importância da otimização, mas também mostram que seu efeito depende do tamanho da entrada e não segue uma tendência estritamente monotônica. As oscilações do fator de aceleração e a maior dispersão observada nos maiores tamanhos de entrada indicam a necessidade de medições adicionais para estudar os mecanismos que contribuem para esse comportamento. Os dados coletados são suficientes para demonstrar o ganho de desempenho, mas não permitem identificar a contribuição de um mecanismo microarquitetural específico para as variações observadas.

Referências

- [1] G. Buttazzo and G. Lipari, “Ptask: An educational C library for programming real-time systems on Linux,” in *2013 IEEE 18th Conference on Emerging Technologies & Factory Automation (ETFA)*, pp. 1–8, 2013.
- [2] T. Hoefer and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–12, 2015.
- [3] C. Curtsinger and E. D. Berger, “Stabilizer: Statistically sound performance evaluation,” *ACM SIGARCH Computer Architecture News*, vol. 41, no. 1, pp. 219–228, 2013.
- [4] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, Version 050, 2026. Disponível em: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel64-and-ia32-architectures-optimization.html>.
- [5] M. Kerrisk and Linux man-pages contributors, “perf_event_open(2) – set up performance monitoring,” *Linux man-pages*, 2026. Disponível em: https://man7.org/linux/man-pages/man2/perf_event_open.2.html.
- [6] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, “Producing wrong data without doing anything obviously wrong!,” *ACM SIGPLAN Notices*, vol. 44, no. 3, pp. 265–276, 2009.
- [7] H. Schildt, *C++: The Complete Reference*. McGraw-Hill, 1998.
- [8] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” in *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, pp. 63–74, 2013.